

Convert this algorithm:

Prompts user to enter number of books

Create an array **books** with size equal number of books this array will take references of book as a numeric value with length **exactly 12 digit**

Create a Boolean array **available** with size equal number of books that will save availability of books

For{0,1,2,...(number of books - 1)}

books[i] = prompts user to enter the reference

prompts user to enter availability of the book

 if availability is 1

 available[i] = true

 else

 available[i] = false

 End if

End For

Number of available = 0

For{0,1,2,...(number of books - 1)}

 if available[i] is true

 display the reference of the book

 number of available +1

 End If

End for

Display the number of available books

Output sample:

Enter number of books : 3

Enter reference of book 1 : 819837405928

Enter availability of the book (1 for available , 0 otherwise) : 1

Enter reference of book 2 : 381947519374

Enter availability of the book (1 for available , 0 otherwise) : 0

Enter reference of book 3 : 184920759275

Enter availability of the book (1 for available , 0 otherwise) : 0

The references of available books :

 819837405928

 184920759275

Number of available books is : 2

```

Scanner sc = new Scanner(System.in)

System.out.print("Enter number of books : ");

int NO_books = sc.nextInt();

long[] books = new long[NO_books];

boolean[] available = new boolean[NO_books];

for(int i = 0; i<NO_books; i++){

    System.out.print("Enter reference of book " + (i+1) + " : ");

    books[i] = sc.nextLong();

    System.out.print("Enter availability of the book ( 1 for available , 0 otherwise) : ");

    int availability = sc.nextInt();

    if( availability == 1)

        available[i] = true;

    else

        available[i] = false;

}

System.out.println("The references of available books : ");

int NO_available = 0;

for(int i = 0; i<NO_books; i++){

    if(available[i]){

        System.out.println("    "+ books[i] );

        NO_available ++;

    }

}

System.out.println("Number of available books is : " + NO_available);

```

Write a program that prompts user to input a **size** of an array and fill it by generate random numbers **between 1 and 10** then display the reverse array , calculate the sum of indicis of odd numbers in the original array

for example, array with size 10 shown as bellow the indicis are the numbers {0,1,2...,9}

2	1	7	4	5	9	3	6	1	9
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

Output sample :

Enter the array size limit : 10

reversed array is:{9,1,6,3,9,5,4,7,1,2}

sum of indicis of odd numbers is : 35

```
Scanner input = new Scanner(System.in);

System.out.print("Enter the array size limit : ");

int size = input.nextInt();

int[] numbers = new int[size];

int sumInd = 0;

for(int i = 0; i<size; i++){

    numbers[i] = (int)( 1+ Math.random() * 10);

    if(numbers[i] %2 != 0)

        sumInd += i;

}

System.out.print( "reversed array is:{" );

for(int i = size-1; i>0; i--){

    System.out.print( numbers[i] + ", " );

}

System.out.println( numbers[0] + "}" );

System.out.println( "sum of indicis of odd numbers is : " + sumInd );
```

Write segment of code:

Prompts the user to enter a password and invoke boolean **isValid()** method that takes **numeric (not String)** parameter to check if the password is palindrome, if it's palindrome return true if it's not return false.

The password is palindrome if it's the same when reading it forwards and backwards

Output sample:

Enter a password: 554434455

554434455 is a valid password

Enter a password: 554434655

554434655 is invalid password

```

public static void main(String [] args){
    Scanner input = new Scanner(System.in);
    System.out.print("Enter a password");
    int password = input.nextInt();

    \\ invoke the method here and show the output

    if(isValid(password))

        System.out.println(password+ " is valid password");

    else

        System.out.println(password+ " is invalid password");

}

```

\\Write isValid method here

```

public static boolean isValid(int pass){

    int orgNum=pass;
    int rev=0;
    while(pass>0){ //Get the reversed number to compare it with the original number
        rev= rev*10 + pass%10;
        pass/=10;
    }
    if(rev==orgNum)
        return true;
    else
        return false;

}

```

Generate a random integer from $10 \leq \text{num} \leq 100$, and print first 30 integers multiplied by 3 starting from the random integer.

Display every 10 integers per line until it reaches 30 integers.

Output sample:

Every 30 integers that are divisible by 3 starting from 12 are:

```
12 15 18 21 24 27 30 33 36 39
42 45 48 51 54 57 60 63 66 69
72 75 78 81 84 87 90 93 96 99
```

Every 30 integers that are divisible by 3 starting from 9 are:

```
9 12 15 18 21 24 27 30 33 36
39 42 45 48 51 54 57 60 63 66
69 72 75 78 81 84 87 90 93 96
```

```
public static void main(String [] args){
    int num=(int)(10+Math.random()*91);
    int Numbers_Per_Line=0;
    int count=0;
    System.out.println("Every 30 integers that are divisible by 3 starting from " +num+ " are:");

    for(int i=num;count<30;i++){

        if(i%3==0){
            System.out.print(i+" ");
            Numbers_Per_Line++;
            count++;
        }

        if(Numbers_Per_Line==10){
            System.out.println();
            Numbers_Per_Line=0;
        }
    }
}
```

Multiple choice :

Q1/

```
public static void main(String[] args){
    String msg = "Hello";
    checkMsg(msg);
    System.out.println(msg);
}
public static void checkMsg(String message){
    if(message == "Hello")
        message = "Empty";
    else
        message = "some";
}
```

Output is:

A-msg

B-Hello

C-Empty

D-some

Q2/

```
String s1 = "abc";
String s2 = "def";
String s3 = s1 + s2;
s1 += s1;
System.out.println(s1); output is:
```

A-abcabc

B-abc

C-abcdef

D-def

Q3/

```
public static void nPrint(String str, int n){
    while(n>0){
        System.out.print(str);
        n--;
    }
}
```

if this method invoked like **nPrint('S',4)** the output will be:

A-S

B-SSS

C-SSSS

D-Error incompatible
type

Q4/

if **takeArray** is a method with **array parameter** and you want to invoke it with an array named **list** so the invoke will be like :

Note : list is an array with size 10

A- takeArray(list[]);

B- tkaeArray(list[9]);

C- takeArray(list);

D- takeArray([] list);

Q5 /

if you have two methods with the same name as these with these headers max(int
n1, int2)

 max(double n1, double n2, double n3)

this called :

A- ambiguous invocation

B- method abstracting

C- pass by reference

D- method overloading

Find the output by tracing the following code:

<pre>public static void main(String[] args){ int i = 1; while(i<=4){ devTwo(i); i++; } } public static void devTwo(int j){ do{ if(j%2!=0){ j--; continue; } else System.out.println(j--); }while(j>1); }</pre>	<u>Output:</u> 2 2 4 2
--	---